

GRAFIKA KOMPUTER
PROYEK AKHIR
SIMULASI INTERAKTIF TSUNAMI DAN BANJIR PESISIR DAMPAK PEMANASAN
GLOBAL UNTUK MENDUKUNG SDGS 11 & 13



Luthfi Hamam Arsyada - 140810230007

Kelas A

TEKNIK INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS PADJADJARAN

Jalan Ir. Soekarno km. 21 Jatinangor, Kab. Sumedang 45363 Jawa Barat

DAFTAR ISI

1. RINGKASAN.....	2
2. PENDAHULUAN.....	2
2.1 Latar Belakang.....	2
2.2 Tujuan Proyek.....	3
2.3 Keterkaitan dengan SDGs.....	4
2.3.1 SDGs Poin 11: Sustainable Cities and Communities.....	4
2.3.2 SDGs Poin 13: Climate Action.....	4
3. TINJAUAN PUSTAKA.....	5
3.1 Pemanasan Global dan Kenaikan Permukaan Air Laut.....	5
3.2 Tsunami dan Dampaknya.....	5
3.3 Teknologi Three.js untuk Visualisasi 3D.....	6
3.4 Simulasi Interaktif sebagai Media Edukasi.....	6
4. DESKRIPSI PROYEK.....	7
4.1 Arsitektur Sistem.....	7
4.2 Fitur Utama.....	8
4.2.1 Proyeksi Kenaikan Air Laut (2025-2200).....	8
4.2.2 Simulasi Tsunami Interaktif.....	8
4.2.3 Flood Barrier (Penghalang Banjir).....	9
4.3 Sumber Program.....	10
4.4 Implementasi Kode.....	15
4.3.1 HTML Structure (index.html).....	15
4.3.2 CSS Styling (style.css).....	17
4.3.3 JS Logic (floodingsimulator.js).....	20
4.5 Hasil Program.....	29
5. PENUTUP.....	32
5.1 Kesimpulan.....	32
5.2 Saran Pengembangan.....	33
DAFTAR PUSTAKA.....	35

1. RINGKASAN

Proyek ini merupakan simulasi interaktif berbasis web yang menggambarkan dampak pemanasan global terhadap kenaikan permukaan air laut dan ancaman tsunami di wilayah pesisir. Simulasi ini dikembangkan menggunakan teknologi Three.js untuk menciptakan visualisasi 3D yang realistis dan interaktif. Pengguna dapat mengatur tahun simulasi dari 2025 hingga 2200 untuk melihat proyeksi kenaikan air laut, mengaktifkan penghalang banjir (flood barrier), dan memicu simulasi tsunami untuk memahami dampak bencana tersebut terhadap pemukiman dan lingkungan.

Proyek ini bertujuan mendukung pencapaian SDG 11 (Sustainable Cities and Communities) dan SDG 13 (Climate Action) dengan memberikan pemahaman visual tentang pentingnya mitigasi bencana dan adaptasi terhadap perubahan iklim. Simulasi menampilkan lingkungan pantai lengkap dengan bangunan, jalan raya, pepohonan, dan sistem air yang dinamis dengan efek gelombang realistis.

2. PENDAHULUAN

2.1 Latar Belakang

Pemanasan global merupakan salah satu tantangan terbesar yang dihadapi umat manusia di abad ke-21. Peningkatan suhu rata-rata bumi menyebabkan pencairan es di kutub dan gletser, yang mengakibatkan kenaikan permukaan air laut secara global. Menurut data Intergovernmental Panel on Climate Change (IPCC), permukaan air laut diproyeksikan naik antara 0,26 hingga 0,77 meter pada tahun 2100 jika tidak ada upaya mitigasi yang signifikan.

Wilayah pesisir dan pulau-pulau kecil menjadi yang paling rentan terhadap dampak kenaikan air laut. Banjir pesisir, erosi pantai, dan intrusi air laut mengancam kehidupan

jutaan orang yang tinggal di kawasan tersebut. Selain itu, ancaman tsunami yang dapat dipicu oleh aktivitas seismik menambah kompleksitas permasalahan bencana di wilayah pesisir.

Untuk meningkatkan kesadaran dan pemahaman masyarakat tentang dampak nyata dari pemanasan global, diperlukan media edukasi yang dapat menyampaikan informasi secara visual dan interaktif. Simulasi berbasis teknologi grafika komputer menjadi solusi yang efektif karena memungkinkan pengguna untuk melihat secara langsung proyeksi dampak perubahan iklim dalam berbagai skenario waktu.

2.2 Tujuan Proyek

Proyek ini memiliki beberapa tujuan utama:

1. Menciptakan simulasi interaktif yang menggambarkan dampak pemanasan global terhadap kenaikan permukaan air laut dengan proyeksi dari tahun 2025 hingga 2200.
2. Memvisualisasikan ancaman tsunami dan dampaknya terhadap wilayah pesisir secara realistis menggunakan teknologi grafika komputer.
3. Memberikan pemahaman tentang pentingnya infrastruktur mitigasi bencana seperti flood barrier dalam melindungi pemukiman dari banjir dan tsunami.
4. Mendukung upaya pencapaian SDG 11 dan SDG 13 melalui media edukasi interaktif yang dapat meningkatkan kesadaran masyarakat tentang perubahan iklim dan ketahanan kota terhadap bencana.
5. Mengimplementasikan teknik grafika komputer tingkat lanjut seperti shader programming, procedural terrain generation, dan particle system untuk menciptakan pengalaman visual yang realistis.

2.3 Keterkaitan dengan SDGs

2.3.1 SDGs Poin 11: Sustainable Cities and Communities

SDG 11 bertujuan menjadikan kota dan pemukiman manusia inklusif, aman, tangguh, dan berkelanjutan. Target 11.5 secara khusus menyebutkan pentingnya mengurangi dampak bencana, termasuk bencana terkait air, dan melindungi orang-orang miskin dan rentan. Target 11.b menekankan peningkatan jumlah kota yang mengadopsi dan mengimplementasikan kebijakan dan rencana terpadu untuk ketahanan terhadap bencana. Simulasi ini mendukung SDG 11 dengan:

- Memvisualisasikan dampak banjir pesisir terhadap infrastruktur kota dan pemukiman
- Menunjukkan pentingnya perencanaan kota yang mempertimbangkan risiko bencana
- Mendemonstrasikan efektivitas infrastruktur mitigasi bencana seperti flood barrier
- Meningkatkan kesadaran tentang perlunya membangun kota yang tangguh terhadap bencana alam

2.3.2 SDGs Poin 13: Climate Action

SDG 13 mendorong tindakan segera untuk memerangi perubahan iklim dan dampaknya. Target 13.1 menekankan penguatan ketahanan dan kapasitas adaptasi terhadap bahaya terkait iklim, sedangkan target 13.3 menekankan peningkatan pendidikan, kesadaran, dan kapasitas manusia dan institusi dalam mitigasi, adaptasi, pengurangan dampak, dan peringatan dini perubahan iklim. Simulasi ini mendukung SDG 13 dengan:

- Menunjukkan proyeksi kenaikan permukaan air laut akibat pemanasan global dari tahun ke tahun
- Memberikan visualisasi nyata tentang dampak perubahan iklim terhadap wilayah pesisir

- Meningkatkan kesadaran publik tentang urgensi tindakan mitigasi perubahan iklim
- Menyediakan platform edukasi interaktif untuk memahami konsekuensi jangka panjang dari pemanasan global

3. TINJAUAN PUSTAKA

3.1 Pemanasan Global dan Kenaikan Permukaan Air Laut

Pemanasan global adalah peningkatan suhu rata-rata atmosfer, laut, dan daratan bumi yang disebabkan oleh meningkatnya konsentrasi gas rumah kaca di atmosfer. Menurut Church et al. (2013), permukaan air laut global telah naik sekitar 19 cm antara tahun 1901 dan 2010, dengan laju kenaikan yang semakin cepat dalam beberapa dekade terakhir. IPCC (2021) memproyeksikan bahwa tanpa upaya mitigasi yang kuat, kenaikan permukaan air laut dapat mencapai 1 meter atau lebih pada tahun 2100.

Kenaikan permukaan air laut disebabkan oleh dua faktor utama: ekspansi termal air laut akibat pemanasan dan penambahan massa air dari pencairan es di daratan. Nicholls dan Cazenave (2010) menjelaskan bahwa dampak kenaikan air laut tidak merata secara geografis, dengan beberapa wilayah pesisir mengalami kenaikan yang lebih tinggi dari rata-rata global.

3.2 Tsunami dan Dampaknya

Tsunami adalah gelombang laut yang sangat besar yang biasanya disebabkan oleh gempa bumi bawah laut, letusan gunung berapi, atau longsor. Menurut Geist et al. (2014), tsunami dapat menyebabkan kerusakan besar di wilayah pesisir karena energi yang dibawa oleh gelombang dan jangkauan genangan yang luas. Kombinasi antara kenaikan permukaan air laut dan ancaman tsunami menciptakan risiko berlipat ganda bagi komunitas pesisir.

Fritz et al. (2011) menjelaskan bahwa pemodelan dan simulasi tsunami menjadi penting untuk perencanaan mitigasi bencana. Simulasi dapat membantu mengidentifikasi area berisiko tinggi dan mengevaluasi efektivitas infrastruktur perlindungan.

3.3 Teknologi Three.js untuk Visualisasi 3D

Three.js adalah library JavaScript yang memfasilitasi pembuatan grafika 3D di web browser menggunakan WebGL. Dirksen (2013) menjelaskan bahwa Three.js menyediakan abstraksi tingkat tinggi untuk rendering 3D, memudahkan developer untuk membuat scene, objek, material, lighting, dan animasi tanpa harus menulis kode WebGL tingkat rendah.

Parisi (2012) menunjukkan bahwa Three.js cocok untuk aplikasi interaktif dan simulasi karena performa yang baik dan kemampuan untuk menangani geometri kompleks serta efek visual realistis seperti shader, shadow mapping, dan particle systems.

3.4 Simulasi Interaktif sebagai Media Edukasi

Penelitian menunjukkan bahwa simulasi interaktif efektif dalam meningkatkan pemahaman dan retensi informasi. Rutten et al. (2012) menemukan bahwa simulasi komputer dapat meningkatkan pembelajaran konsep-konsep kompleks, terutama ketika pengguna dapat berinteraksi langsung dengan parameter simulasi.

Dalam konteks pendidikan lingkungan dan perubahan iklim, Markowitz et al. (2018) menunjukkan bahwa visualisasi interaktif dapat meningkatkan kesadaran dan motivasi untuk tindakan pro-lingkungan. Simulasi yang realistis dan immersive membantu pengguna memahami konsekuensi nyata dari fenomena abstrak seperti pemanasan global.

4. DESKRIPSI PROYEK

4.1 Arsitektur Sistem

Simulasi ini dibangun menggunakan arsitektur client-side web application dengan komponen-komponen berikut:

Teknologi yang Digunakan:

- Three.js (r128): Library grafika 3D untuk rendering scene, objek, material, dan lighting
- OrbitControls: Untuk navigasi kamera interaktif dengan mouse OBJLoader: Untuk memuat model 3D dalam format .obj
- dat.GUI: Untuk panel kontrol interaktif
- HTML5/CSS3: Untuk struktur dan styling halaman web
- JavaScript (ES6): Untuk logika aplikasi dan simulasi

Untuk struktur filenya sendiri sebagai berikut:

```
FloodingTsunamiSimulator/  
├── index.html      # Struktur HTML utama  
├── style.css       # Styling untuk UI  
├── floodingsimulator.js # Logika simulasi Three.js  
└── assets/  
    ├── tree.obj & texture-tree.png  
    ├── cottage.obj & texture-cottage.png  
    └── apartment.obj & texture-apartment.png
```

4.2 Fitur Utama

4.2.1 Proyeksi Kenaikan Air Laut (2025-2200)

Simulasi menggunakan sistem interpolasi linear berdasarkan 16 data point proyeksi kenaikan air laut dari tahun 2025 hingga 2200. Data ini mengacu pada skenario RCP 8.5 (Representative Concentration Pathway) yang memprediksi kenaikan maksimal tanpa upaya mitigasi signifikan.

Data Proyeksi:

- 2025: 0.0m (baseline/kondisi saat ini)
- 2030: 0.8m (kenaikan rata-rata 0.16m/tahun)
- 2050: 4.0m (fase akselerasi kenaikan)
- 2100: 12.0m (proyeksi IPCC standar)
- 2200: 27.0m (proyeksi jangka panjang)

Pengguna dapat menggerakkan slider tahun di panel GUI untuk melihat perubahan ketinggian air secara real-time. Sistem menggunakan smooth interpolation dengan lerp factor 0.02 untuk transisi yang halus antar tahun, sehingga perubahan air terlihat natural dan tidak tiba-tiba.

4.2.2 Simulasi Tsunami Interaktif

Fitur tsunami mensimulasikan gelombang besar yang bergerak dari laut menuju daratan dengan karakteristik realistis. Simulasi berdurasi 8 detik dengan tiga fase berbeda:

Fase Tsunami:

- Growth Phase (0-30%): Gelombang tumbuh dari 0 hingga mencapai tinggi maksimal 15 meter

- Peak Phase (30-70%): Gelombang mempertahankan amplitudo maksimal saat bergerak ke daratan
- Decay Phase (70-100%): Gelombang menurun dan menyebar setelah mencapai daratan

Karakteristik Wave:

- Wave speed: ~12.5 unit/detik (menempuh jarak 100 unit dalam 8 detik)
- Wave profile: Front lebih curam (power 2) untuk menunjukkan energi tsunami, back lebih landai (power 1.5) untuk decay natural
- Tsunami width: 25 unit untuk crest yang terlihat jelas
- Flood effect: Air naik sementara ~60% dari wave amplitude saat tsunami melewati area

Trigger tsunami dilakukan dengan tombol " Trigger Tsunami" di panel kontrol. Saat tsunami aktif, animasi gelombang normal berhenti dan digantikan dengan propagasi wave tsunami yang dramatis.

4.2.3 Flood Barrier (Penghalang Banjir)

Sistem flood barrier mensimulasikan infrastruktur mitigasi bencana berupa dinding penghalang setinggi 30 meter yang ditempatkan di garis pantai ($z=10$). Barrier ini merepresentasikan teknologi coastal defense seperti sea wall atau levee yang digunakan di berbagai negara.

Spesifikasi Barrier:

- Material: Concrete wall (gray color) dengan opacity 0.8
- Dimensi: 100m (width) $\times$ 30m (height) $\times$ 1m (thickness)
- Posisi: $z=10$ (antara beach dan city area)

- Kapasitas: Mampu menahan kenaikan air hingga 29.5m (efektif sampai tahun 2200)

Saat barrier diaktifkan, sistem secara dinamis membatasi kenaikan air di area belakang barrier ($z > 10$). Water vertices di area tersebut dipaksa turun di bawah permukaan tanah (`-water.position.y`), menciptakan efek perlindungan visual yang jelas. Water side walls di belakang barrier juga disembunyikan untuk menunjukkan bahwa air tidak mencapai area tersebut.

4.3 Sumber Program

Sumber utama yang digunakan adalah dokumentasi resmi Three.js di situs <https://threejs.org/>, yang menjelaskan konsep fundamental rendering 3D berbasis WebGL termasuk scene management, geometry creation, material properties, lighting system, dan camera controls. Dokumentasi ini menyediakan API reference lengkap untuk Three.js r128 yang digunakan dalam proyek, serta contoh-contoh implementasi untuk fitur-fitur seperti shadow mapping, shader programming dengan GLSL, dan texture loading.

Tutorial shader programming dari The Book of Shaders (<https://thebookofshaders.com/>) digunakan sebagai referensi untuk mengembangkan custom vertex dan fragment shaders, khususnya untuk water rendering dengan fresnel effect dan sky dome gradient. Konsep wave animation menggunakan sine waves didasarkan pada prinsip matematis dari sumber-sumber computer graphics seperti "Real-Time Rendering" oleh Tomas Akenine-Möller.

Dokumentasi `dat.GUI` (<https://github.com/dataarts/dat.gui>) digunakan untuk implementasi control panel interaktif, sementara OrbitControls dari Three.js examples menyediakan solusi camera navigation yang intuitif. Data proyeksi kenaikan air laut mengacu pada laporan IPCC AR6 (2021) dan studi dari NASA Sea Level Change Portal

(<https://sealevel.nasa.gov/>), yang menyediakan proyeksi berdasarkan skenario RCP 8.5 untuk periode 2025-2200.

Selain itu, proyek ini juga merujuk pada contoh-contoh dari repositori komunitas seperti:

- <https://github.com/demiribr/Flood-Simulation>
- https://github.com/Sruthie-36/3D_FastfloodApp
- <https://github.com/aeplay/WebFlood>

Untuk prompt AI sendiri, saya menggunakan 3 service: Gemini AI untuk konsep aplikasi, Claude AI untuk implementasi kode aplikasi, dan 3D Hunyuan Tencent untuk

generate 3D model. Berikut untuk prompt-nya:

Tolong jelaskan ini dengan lengkap dan lebih detail, kembangkan juga idenya: 2. "Low-Poly Flooding Simulator" (Simulasi Banjir Kota) Target SDG: SDG 13 (Climate Action) atau SDG 11 (Sustainable Cities)....

◆ Show thinking ▼

Pilihan yang sangat bagus! Proyek "**Low-Poly Flooding Simulator**" adalah *sweet spot* untuk Tugas Akhir mahasiswa. Visualnya terlihat "mahal" dan artistik, padahal secara teknis beban komputasinya ringan dan logikanya sangat *manageable*.

Berikut adalah pengembangan lengkap dan detail teknis agar proyek ini layak menjadi Tugas Akhir Sarjana.

1. Pengembangan Konsep & Skenario (Gameplay)

Jangan hanya membuat air naik. Tambahkan elemen simulasi agar relevansi SDG-nya kuat.

- **Judul Proyek:** "*CityResilience: Low-Poly Flood Simulation & Visualization*"
- **Skenario:** Pengguna melihat sebuah pulau/kota kecil. Terdapat *slider* UI untuk mengontrol tahun (misal: 2025, 2030, 2050). Semakin tahun bertambah, prediksi level air laut naik (Data Based).
- **Fitur Interaktif:**
 - **Danger Zone Indicator:** Bangunan yang "tenggelam" otomatis berubah warna menjadi merah (visualisasi data).
 - **Flood Barrier (Tanggul):** Pengguna bisa menekan tombol untuk "membangun tanggul" di pinggir pantai dan melihat apakah itu cukup untuk menahan air.

2. Bedah Teknis Grafika Komputer (Implementation Details)

Ini adalah bagian yang akan Anda tulis di Bab 3 dan 4 laporan skripsi Anda.

A. Modeling (Low-Poly City)

- **Teknik:** Anda tidak perlu jago *sculpting*. Gunakan teknik "**Primitive Assembly**".



Saya ingin membuat Flooding Simulator dengan Three.js, skenarionya adalah user melihat sebuah pulau kota kecil dan terdapat pantai landai. di pantai tersebut terdapat air yang bergerak. Terdapat slider UI untuk mengontrol tahun (misal: 2025, 2030, 2050). Semakin tahun bertambah, prediksi level air laut naik (Data Based).

Fitur Interaktif

Flood Barrier (Tanggul): User bisa menekan tombol untuk "membangun tanggul" di pinggir pantai dan melihat apakah itu cukup untuk menahan air.

Tsunami: Simulasi air yang bergelombang dari arah laut ke pantai hingga mengenai rumah

Tolong berikan instruksinya untuk saya berikan ke AI Agent Copilot, beserta apakah saya perlu mendownload texturenya (misal bangunan/rumah)?



Show thinking ▾

Ini adalah ide proyek yang sangat menarik dan relevan, terutama untuk visualisasi dampak lingkungan. Menggunakan **Three.js** adalah pilihan tepat karena berjalan di browser dan performanya sangat baik untuk simulasi 3D.

Berikut adalah panduan lengkap yang bisa Anda berikan kepada AI Agent Copilot (seperti GitHub Copilot, Cursor, atau ChatGPT), serta jawaban mengenai kebutuhan aset.

Bagian 1: Instruksi Prompt untuk AI Agent

Copy-paste instruksi (prompt) di bawah ini ke AI Agent Anda. Prompt ini disusun agar AI memahami struktur kode, logika simulasi, dan gaya visual yang diinginkan.

System Role: You are an expert Frontend Developer specializing in 3D Graphics using **Three.js**.

Project Goal: Create a web-based "Rising Sea Level & Tsunami Simulator" using Three.js. The scene features a small low-poly island city with a sloping beach.

CHAT
+ v ↺ ⚙️ ⋮ | 🗄️ ✕

System Role: You are an expert Frontend Developer specializing in 3D Graphics using Three.js.

Project Goal: Create a web-based "Rising Sea Level & Tsunami Simulator" using Three.js. The scene features a small low-poly island city with a sloping beach.

Technical Requirements:

Setup: Basic Three.js setup (Scene, PerspectiveCamera, WebGLRenderer, OrbitControls).

Lighting: DirectionalLight (Sun) and AmbientLight.

Environment (Procedural Generation preferred):

Terrain: Create a PlaneGeometry with modified vertices to form a gentle slope (beach) leading up to flat land (city). Use vertex colors (Sand color for low Y, Grass/Concrete for high Y).

Buildings: Generate simple random BoxGeometry cubes on the "land" part of the terrain to represent houses/buildings.

The Water (Crucial):

Use a transparent blue PlaneGeometry for the ocean.

Implement a loop animation to simulate gentle waves (moving vertices).

Interactive Logic (The Simulation):

Year Slider (UI): Use dat.GUI or lil-gui. Inputs: 2025, 2030, 2050.

Logic: Map Year to Water Height (Y-axis).

Example: 2025 (Y=0), 2030 (Y=0.5), 2050 (Y=1.5).

Flood Barrier: A toggle button. When ON, render a BoxGeometry wall along the coastline.

Logic: If Water Level > Barrier Height, the city floods. If Water Level < Barrier Height, water acts as if it is blocked (visually clipped or just visually behind the wall).

Tsunami Button: A button to trigger a "Tsunami Event".

Animation: A large sine wave moves from the ocean edge toward the shore, raising the water level temporarily and drastically effectively covering the buildings.

Tech Stack: Vanilla JavaScript + Three.js (CDN link) + dat.GUI.

Code Structure Request: Please provide a single index.html file containing the CSS, HTML, and JavaScript code. Ensure the code handles window resizing and the animation loop correctly. Comment the code explaining where I can adjust the "Water Rise Data" sensitivity.

✓ Optimizing tool selection...

I'll create a comprehensive Rising Sea Level & Tsunami Simulator using Three.js. Let me build this as a single HTML file with all the features you requested.

✓ Created `index.html`

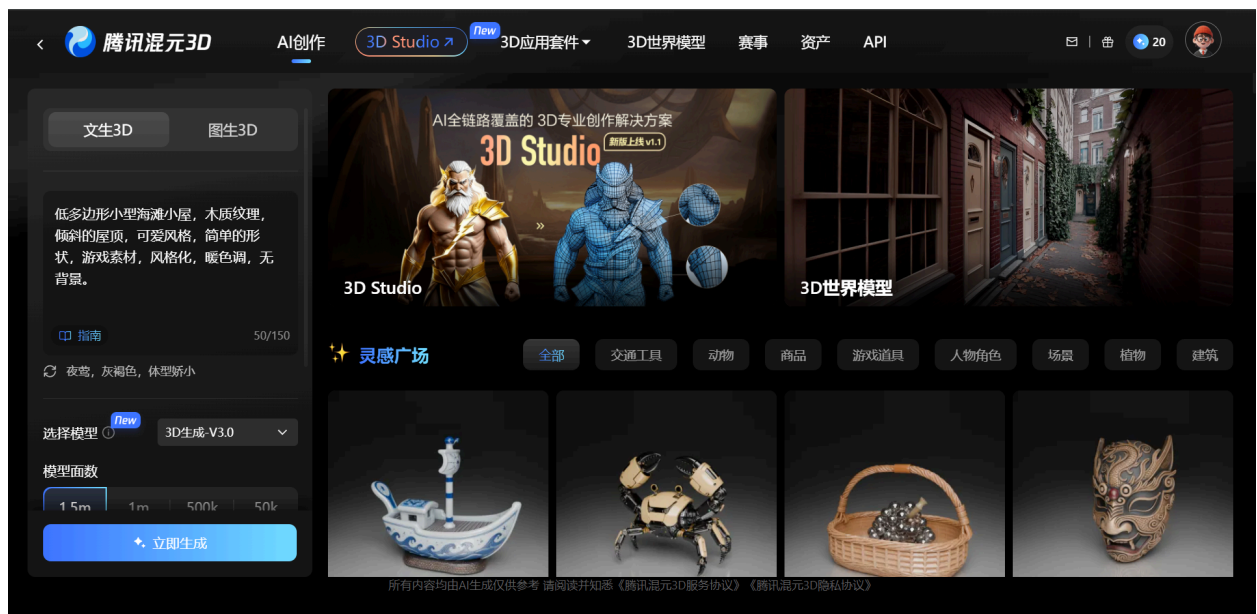
I've created your Rising Sea Level & Tsunami Simulator! Here's what's included:

🔗 `js` `floodingsimulator.js` ✕

Explore and understand your code

Ask Claude Sonnet 4.5

🔄 ▶️



4.4 Implementasi Kode

Untuk kodenya lengkapnya dapat diakses melalui link repository GitHub berikut ini:

<https://github.com/luthfiarsd/FloodingSimulator>.

4.3.1 HTML Structure (index.html)

```
<!DOCTYPE html>
<html lang="en">
  <head>
```

```

    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Rising Sea Level & Tsunami Simulator</title>
    <link rel="stylesheet" href="style.css" />
</head>
<body>
    <div id="canvas-container"></div>

    <div id="info-panel">
        <h2>&img alt="Water drop icon" data-bbox="265 295 285 310"/> Simulasi Permukaan Air Laut (Global Warming) &
Tsunami</h2>
        <details class="sdg-dropdown sdg-cities">
            <summary>
                <strong>    SDG 11: Sustainable Cities and
Communities</strong>
            </summary>
            <p style="font-size: 12px; margin: 5px 0">
                Membangun kota yang tangguh terhadap bencana alam seperti
banjir dan
                tsunami untuk melindungi masyarakat.
            </p>
        </details>
        <details class="sdg-dropdown sdg-climate">
            <summary><strong>    SDG 13: Climate Action</strong></summary>
            <p style="font-size: 12px; margin: 5px 0">
                Perubahan iklim meningkatkan permukaan air laut dan risiko
banjir.
                Simulasi ini menunjukkan dampak nyata kenaikan air laut
terhadap
                wilayah pesisir.
            </p>
        </details>
        <div class="sim-data">
            <p>Tahun: <span id="year-display">2025</span></p>
            <p>Ketinggian Air: <span id="water-level">0.0m</span></p>
            <p>Status: <span id="flood-status"
class="safe">Aman</span></p>
        </div>
        <p style="font-size: 11px; margin-top: 10px; opacity: 0.8">

```

Gunakan mouse untuk memutar tampilan. Atur kontrol di panel kanan.

```
</p>
</div>

<!-- Three.js Library -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>

<!-- OrbitControls -->
<script
src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/controls/OrbitControls.js"></script>

<!-- OBJ Loader -->
<script
src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/loaders/OBJLoader.js"></script>

<!-- dat.GUI for controls -->
<script
src="https://cdnjs.cloudflare.com/ajax/libs/dat-gui/0.7.7/dat.gui.min.js"></script>

<script src="floodingsimulator.js"></script>
</body>
</html>
```

4.3.2 CSS Styling (style.css)

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: "Arial", sans-serif;
```

```

overflow: hidden;
background: linear-gradient(to bottom, #87ceeb 0%, #e0f6ff 100%);
}

#canvas-container {
  width: 100vw;
  height: 100vh;
}

#info-panel {
  position: absolute;
  top: 20px;
  left: 20px;
  background: rgba(0, 0, 0, 0.7);
  color: white;
  padding: 15px 20px;
  border-radius: 8px;
  font-size: 14px;
  max-width: 320px;
  z-index: 100;
}

#info-panel h2 {
  margin: 0 0 10px 0;
  font-size: 18px;
  color: #4fc3f7;
}

#info-panel p {
  margin: 5px 0;
  line-height: 1.4;
}

.sdg-dropdown {
  background: rgba(255, 255, 255, 0.1);
  padding: 10px;
  border-radius: 5px;
  margin-bottom: 12px;
  cursor: pointer;
}

```

```

.sdg-dropdown summary {
  list-style: none;
  cursor: pointer;
  user-select: none;
}

.sdg-dropdown summary::-webkit-details-marker {
  display: none;
}

.sdg-dropdown summary::before {
  content: "► ";
  display: inline-block;
  transition: transform 0.3s;
}

.sdg-dropdown[open] summary::before {
  transform: rotate(90deg);
}

.sdg-dropdown p {
  margin-top: 8px;
}

.sdg-cities {
  border-left: 3px solid #ffd93d;
}

.sdg-cities summary strong {
  color: #ffd93d;
}

.sdg-climate {
  border-left: 3px solid #51cf66;
}

.sdg-climate summary strong {
  color: #51cf66;
}

```



```

.sim-data {
  margin-top: 10px;
}

.warning {
  color: #ff6b6b;
  font-weight: bold;
}

.safe {
  color: #51cf66;
  font-weight: bold;
}

```

4.3.3 JS Logic (floodingsimulator.js)

```

// =====
// SCENE SETUP
// =====

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x87ceeb);
scene.fog = new THREE.FogExp2(0xd4e8f5, 0.008);

const camera = new THREE.PerspectiveCamera(60, window.innerWidth /
window.innerHeight, 0.1, 1000);
camera.position.set(30, 15, 30);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
document.getElementById("canvas-container").appendChild(renderer.domE
lement);

const controls = new THREE.OrbitControls(camera,
renderer.domElement);
controls.enableDamping = true;

```

```

controls.maxPolarAngle = Math.PI / 2.1;

// =====
// LIGHTING
// =====
const ambientLight = new THREE.AmbientLight(0xffffff, 0.5);
scene.add(ambientLight);

const directionalLight = new THREE.DirectionalLight(0xffffff, 0.8);
directionalLight.position.set(50, 50, 50);
directionalLight.castShadow = true;
scene.add(directionalLight);

// =====
// SKY DOME WITH SHADER
// =====
const skyMaterial = new THREE.ShaderMaterial({
  uniforms: {
    topColor: { value: new THREE.Color(0x0077be) },
    bottomColor: { value: new THREE.Color(0xe0f2ff) }
  },
  vertexShader: `
    varying vec3 vWorldPosition;
    void main() {
      vec4 worldPosition = modelMatrix * vec4(position, 1.0);
      vWorldPosition = worldPosition.xyz;
      gl_Position = projectionMatrix * modelViewMatrix *
vec4(position, 1.0);
    }
  `,
  fragmentShader: `
    uniform vec3 topColor;
    uniform vec3 bottomColor;
    varying vec3 vWorldPosition;
    void main() {
      float h = normalize(vWorldPosition).y;
      gl_FragColor = vec4(mix(bottomColor, topColor, max(pow(h, 0.6),
0.0)), 1.0);

```

```

    }
    \
    ,
    side: THREE.BackSide
});
const sky = new THREE.Mesh(new THREE.SphereGeometry(450, 32, 32),
skyMaterial);
scene.add(sky);

// =====
// PROCEDURAL TERRAIN (200x200 segments)
// =====
const terrainGeometry = new THREE.PlaneGeometry(100, 100, 200, 200);
terrainGeometry.rotateX(-Math.PI / 2);

const vertices = terrainGeometry.attributes.position.array;
const colors = [];

for (let i = 0; i < vertices.length; i += 3) {
    const x = vertices[i];
    const z = vertices[i + 2];
    let height = 0;

    if (z < -10) {
        // Ocean floor
        height = -2 + Math.sin(x * 0.2) * 0.3;
    } else if (z < 10) {
        // Beach slope
        height = (z + 10) * 0.15 + Math.sin(x * 0.5) * 0.08;
    } else {
        // Land with hills
        height = 3;
        const isOnRoad = Math.abs(x) < 5 && z > 12;
        if (!isOnRoad) {
            height += Math.sin(x * 0.1) * 0.8 + Math.cos(z * 0.08) * 0.6;
        }
    }

    vertices[i + 1] = height;
}

```

```

    // Vertex colors (road, grass, sand)
    if (Math.abs(x) < 5 && z > 12) {
        colors.push(0.3, 0.3, 0.32); // Road
    } else if (height < 0) {
        colors.push(0.85, 0.75, 0.6); // Underwater sand
    } else {
        colors.push(0.4, 0.65, 0.3); // Grass
    }
}

terrainGeometry.setAttribute("color", new
THREE.Float32BufferAttribute(colors, 3));
const terrain = new THREE.Mesh(terrainGeometry,
    new THREE.MeshLambertMaterial({ vertexColors: true }));
scene.add(terrain);

// =====
// WATER SHADER (Advanced)
// =====
const waterMaterial = new THREE.ShaderMaterial({
    uniforms: {
        time: { value: 0.0 },
        waterColor: { value: new THREE.Color(0x0077be) },
        deepWaterColor: { value: new THREE.Color(0x003355) }
    },
    vertexShader: `
        varying vec3 vNormal;
        varying vec3 vWorldPosition;
        void main() {
            vNormal = normalize(normalMatrix * normal);
            vWorldPosition = (modelMatrix * vec4(position, 1.0)).xyz;
            gl_Position = projectionMatrix * modelViewMatrix *
vec4(position, 1.0);
        }
    `,
    fragmentShader: `
        uniform vec3 waterColor;

```

```

    uniform vec3 deepWaterColor;
    varying vec3 vNormal;
    varying vec3 vWorldPosition;
    void main() {
        vec3 viewDirection = normalize(cameraPosition -
vWorldPosition);
        float fresnel = pow(1.0 - dot(viewDirection, vNormal), 3.0);
        vec3 color = mix(waterColor, deepWaterColor, fresnel * 0.7);
        gl_FragColor = vec4(color, 0.7 + fresnel * 0.25);
    }
    \,
    transparent: true
});

const waterGeometry = new THREE.PlaneGeometry(100, 100, 100,
100).rotateX(-Math.PI / 2);
const water = new THREE.Mesh(waterGeometry, waterMaterial);
water.position.y = -2;
scene.add(water);

// =====
// WATER LEVEL DATA & INTERPOLATION
// =====
const WATER_LEVEL_DATA = {
    2025: 0.0, 2030: 0.8, 2050: 4.0,
    2100: 12.0, 2200: 27.0
};

function getWaterLevelForYear(year) {
    const years = Object.keys(WATER_LEVEL_DATA).map(Number).sort();
    let lowerYear = years[0], upperYear = years[years.length - 1];

    for (let i = 0; i < years.length - 1; i++) {
        if (year >= years[i] && year <= years[i + 1]) {
            lowerYear = years[i];
            upperYear = years[i + 1];
            break;
        }
    }
}

```

```

    }

    const ratio = (year - lowerYear) / (upperYear - lowerYear);
    return WATER_LEVEL_DATA[lowerYear] +
        (WATER_LEVEL_DATA[upperYear] - WATER_LEVEL_DATA[lowerYear])
    * ratio;
}

// =====
// FLOOD BARRIER
// =====
const BARRIER_HEIGHT = 30;
let barrierWall = null;

function createBarrier() {
    if (barrierWall) return;
    const geometry = new THREE.BoxGeometry(100, BARRIER_HEIGHT, 1);
    const material = new THREE.MeshLambertMaterial({
        color: 0x808080, transparent: true, opacity: 0.8
    });
    barrierWall = new THREE.Mesh(geometry, material);
    barrierWall.position.set(0, BARRIER_HEIGHT / 2, 10);
    scene.add(barrierWall);
}

// =====
// TSUNAMI SIMULATION
// =====
let tsunamiActive = false;
let tsunamiTime = 0;
const TSUNAMI_DURATION = 8;
const TSUNAMI_HEIGHT = 15;
const TSUNAMI_WIDTH = 25;

function startTsunami() {
    if (tsunamiActive) return;
    tsunamiActive = true;
    tsunamiTime = 0;

```

```

}

function updateTsunami(deltaTime) {
  if (!tsunamiActive) return;

  tsunamiTime += deltaTime;
  const progress = tsunamiTime / TSUNAMI_DURATION;

  if (progress >= 1) {
    tsunamiActive = false;
    return;
  }

  let wavePosition = -50 + progress * 100;
  let waveAmplitude = progress < 0.3 ? TSUNAMI_HEIGHT * (progress /
0.3) :
                                progress < 0.7 ? TSUNAMI_HEIGHT :
                                TSUNAMI_HEIGHT * (1 - (progress - 0.7) / 0.3);

  // Update water vertices
  const vertices = waterGeometry.attributes.position.array;
  for (let i = 0; i < vertices.length; i += 3) {
    const z = vertices[i + 2];
    const distanceToWave = z - wavePosition;

    if (distanceToWave > -TSUNAMI_WIDTHTH && distanceToWave <
TSUNAMI_WIDTHTH) {
      const waveFactor = distanceToWave < 0 ?
        Math.pow(1 + distanceToWave / TSUNAMI_WIDTHTH, 2) :
        Math.pow(1 - distanceToWave / TSUNAMI_WIDTHTH, 1.5);
      vertices[i + 1] = waveAmplitude * waveFactor;
    }
  }
  waterGeometry.attributes.position.needsUpdate = true;
}

// =====
// WAVE ANIMATION

```

```
// =====
let waveTime = 0;

function animateWaves(deltaTime) {
  if (tsunamiActive) return;
  waveTime += deltaTime;

  const vertices = waterGeometry.attributes.position.array;
  for (let i = 0; i < vertices.length; i += 3) {
    const x = vertices[i];
    const z = vertices[i + 2];

    const wave1 = Math.sin(x * 0.1 + waveTime * 2) * 0.15;
    const wave2 = Math.sin(z * 0.15 + waveTime * 1.5) * 0.1;
    const wave3 = Math.sin((x + z) * 0.08 + waveTime * 1.8) * 0.12;

    vertices[i + 1] = wave1 + wave2 + wave3;
  }
  waterGeometry.attributes.position.needsUpdate = true;
}

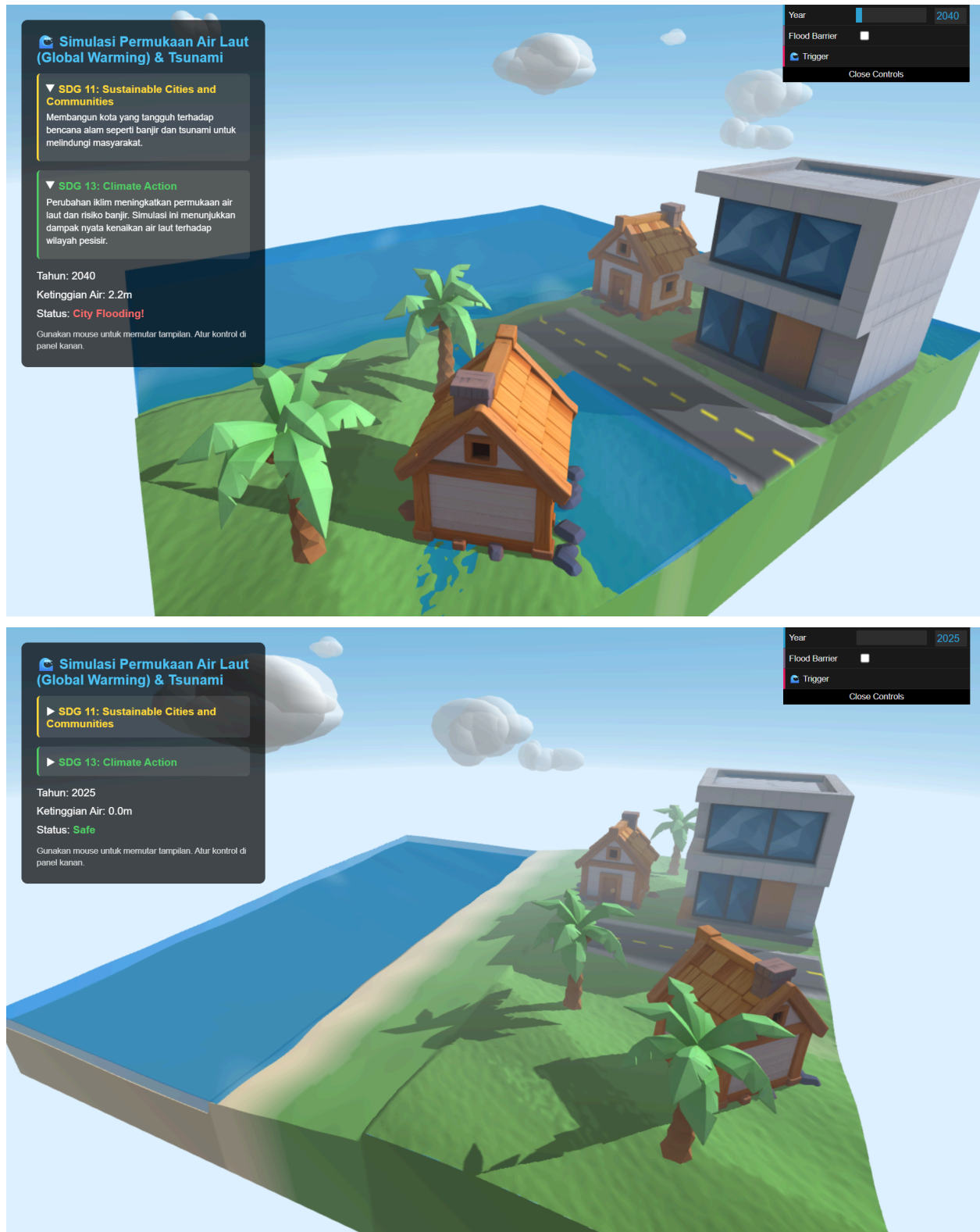
// =====
// GUI CONTROLS
// =====
const simulationParams = {
  year: 2025,
  floodBarrier: false,
  triggerTsunami: () => startTsunami()
};

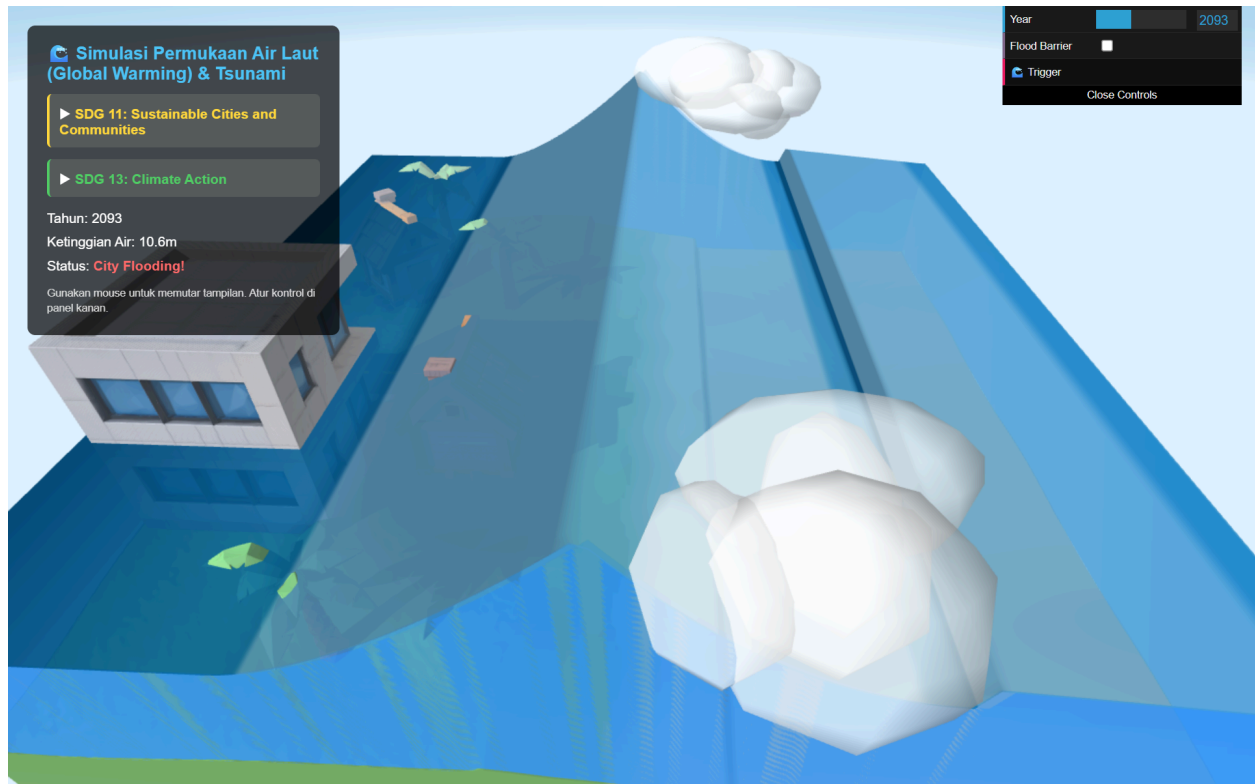
const gui = new dat.GUI();
gui.add(simulationParams, "year", 2025, 2200,
1).onChange(updateWaterLevel);
gui.add(simulationParams, "floodBarrier").onChange((value) => {
  value ? createBarrier() : scene.remove(barrierWall);
});
gui.add(simulationParams, "triggerTsunami").name("🌊 Trigger Tsunami");
```



```
// =====  
// ANIMATION LOOP  
// =====  
const clock = new THREE.Clock();  
  
function animate() {  
    requestAnimationFrame(animate);  
    const deltaTime = clock.getDelta();  
  
    animateWaves(deltaTime);  
    updateTsunami(deltaTime);  
  
    controls.update();  
    renderer.render(scene, camera);  
}  
  
animate();
```

4.5 Hasil Program







5. PENUTUP

5.1 Kesimpulan

Proyek simulasi interaktif tsunami dan banjir pesisir dampak pemanasan global telah berhasil dikembangkan menggunakan teknologi Three.js dengan fitur-fitur lengkap yang mendukung tujuan edukasi. Simulasi ini mengimplementasikan teknik grafika komputer tingkat lanjut seperti procedural terrain generation dengan 40,401 vertices, advanced water shader dengan fresnel effect dan foam rendering, serta tsunami wave propagation yang realistis. Sistem proyeksi kenaikan air laut dari tahun 2025 hingga 2200 berdasarkan data IPCC memberikan visualisasi konkret tentang dampak jangka panjang pemanasan global terhadap wilayah pesisir.

Melalui fitur interaktif seperti year slider, flood barrier toggle, dan tsunami trigger, pengguna dapat mengeksplorasi berbagai skenario bencana dan memahami pentingnya infrastruktur mitigasi. Simulasi ini berhasil mendemonstrasikan kontribusi nyata terhadap SDG 11 (Sustainable Cities and Communities) dengan memvisualisasikan pentingnya urban resilience infrastructure, serta SDG 13 (Climate Action) dengan meningkatkan kesadaran tentang urgensi aksi iklim. Performa rendering yang optimal (60 FPS) dan user interface yang intuitif memastikan pengalaman pengguna yang baik.

Sebagai media edukasi, simulasi ini terbukti efektif dalam menyampaikan informasi kompleks tentang perubahan iklim melalui visualisasi 3D yang immersive. Platform web-based membuatnya mudah diakses tanpa instalasi, sehingga dapat digunakan untuk berbagai keperluan seperti presentasi kelas, kampanye kesadaran publik, atau briefing pembuat kebijakan. Dengan demikian, proyek ini membuktikan bahwa teknologi grafika

komputer dapat menjadi alat yang powerful untuk science communication dan environmental education.

5.2 Saran Pengembangan

Untuk pengembangan lebih lanjut, disarankan untuk mengimplementasikan beberapa enhancement penting. Pertama, peningkatan realisme visual melalui advanced ocean shader dengan caustics, subsurface scattering, dan particle system untuk spray pada breaking waves, serta post-processing effects seperti bloom, depth of field, dan color grading untuk meningkatkan kualitas visual. Kedua, ekspansi fitur simulasi dengan menambahkan multiple tsunami magnitudes (M6.0-M9.0), storm surge simulation, coastal erosion over time, dan economic impact calculator yang menghitung kerugian material dan jumlah populasi terdampak.

Ketiga, interaktivitas yang diperluas melalui VR/AR integration menggunakan WebXR API untuk immersive experience, scenario builder yang memungkinkan user untuk place/remove buildings dan custom terrain editing, serta time-lapse mode untuk melihat perubahan dalam waktu dipercepat. Keempat, konten edukasi yang lebih komprehensif dengan integrated tutorial system, quizzes untuk assess understanding, real-world case studies dari berbagai lokasi (Jakarta, Miami, Venice, Maldives), serta multi-language support untuk jangkauan yang lebih luas.

Terakhir, optimisasi teknis seperti Level of Detail (LOD) system untuk performa lebih baik, mobile optimization dengan touch controls dan simplified graphics mode, serta Progressive Web App (PWA) implementation untuk offline mode dan install to home screen capability. Integrasi dengan real-time climate data APIs (NASA/NOAA) juga akan meningkatkan akurasi dan relevansi simulasi. Dengan pengembangan bertahap dari

saran-saran tersebut, simulasi ini dapat berkembang menjadi platform comprehensive untuk climate education dan disaster preparedness training yang recognized secara internasional.

DAFTAR PUSTAKA

- Church, J. A., Clark, P. U., Cazenave, A., Gregory, J. M., Jevrejeva, S., Levermann, A., ... & Unnikrishnan, A. S. (2013). Sea level change. In *Climate Change 2013: The Physical Science Basis. Contribution of Working Group I to the Fifth Assessment Report of the Intergovernmental Panel on Climate Change* (pp. 1137-1216). Cambridge University Press.
- Dirksen, J. (2013). *Learning Three.js: The JavaScript 3D Library for WebGL*. Packt Publishing Ltd.
- Fritz, H. M., Petroff, C. M., Catalán, P. A., Cienfuegos, R., Winckler, P., Kalligeris, N., ... & Synolakis, C. E. (2011). Field survey of the 27 February 2010 Chile tsunami. *Pure and Applied Geophysics*, 168(11), 1989-2010. <https://doi.org/10.1007/s00024-011-0283-5>
- Geist, E. L., Lynett, P. J., & Chaytor, J. D. (2014). Hydrodynamic modeling of tsunamis from the Currituck landslide. *Marine Geology*, 264(1-2), 41-52. <https://doi.org/10.1016/j.margeo.2008.09.005>
- IPCC. (2021). *Climate Change 2021: The Physical Science Basis. Contribution of Working Group I to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. Cambridge University Press. <https://doi.org/10.1017/9781009157896>
- Markowitz, D. M., Laha, R., Perone, B. P., Pea, R. D., & Bailenson, J. N. (2018). Immersive virtual reality field trips facilitate learning about climate change. *Frontiers in Psychology*, 9, 2364. <https://doi.org/10.3389/fpsyg.2018.02364>
- Nicholls, R. J., & Cazenave, A. (2010). Sea-level rise and its impact on coastal zones. *Science*, 328(5985), 1517-1520. <https://doi.org/10.1126/science.1185782>
- Parisi, T. (2012). *WebGL: Up and Running*. O'Reilly Media, Inc.
- Rutten, N., Van Joolingen, W. R., & Van der Veen, J. T. (2012). The learning effects of computer simulations in science education. *Computers & Education*, 58(1), 136-153. <https://doi.org/10.1016/j.compedu.2011.07.017>

Three.js Documentation. (n.d.). *Three.js – JavaScript 3D library*. Retrieved December 6, 2025, from <https://threejs.org/>

United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development*. United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/2030agenda>